Pool-ball Detection and Recognition

Manon Yiqin Galian Ng (23375206) – CSU44053

1 Overview

The objective of this project was to locate pool balls in photographs of a table and identify each ball by type and number. I divided the task into detecting candidate balls and recognising their identities, then converted their positions into table coordinates for the final views. The system was implemented using OpenCV 4.12.0 and C++17.

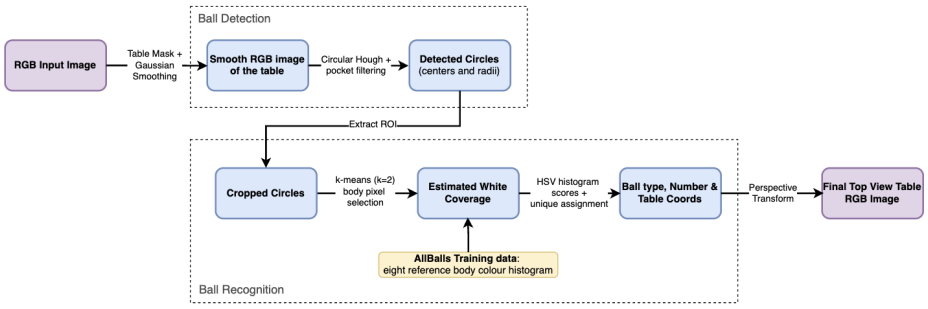


Figure 1: Diagram of the Final Pipeline

2 Detailed Design

2.1 Ball Detection

I used the supplied corners to create a mask of the playing surface and set the background to black. I then converted the image to greyscale and smoothed it with a 5×5 Gaussian filter ($\sigma = 1.5$) to reduce small intensity variations before circular Hough transform.

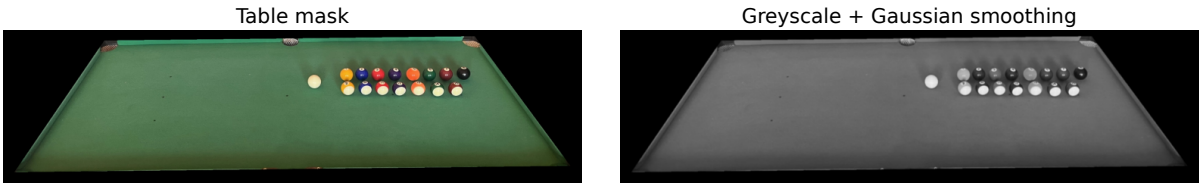


Figure 2: Masking (left) and greyscale smoothing (right) on AllBalls.

I initially varied Hough parameters manually on AllBalls. This was slow and configurations were hard to compare, so I evaluated 480 configurations. The centers were matched to annotations in table coordinates. I ranked settings by highest number of correct detections (missed ball cannot be recognised later) then by fewest extras and finally by lowest centre error (smallest average distance between detected and annotated ball centres).

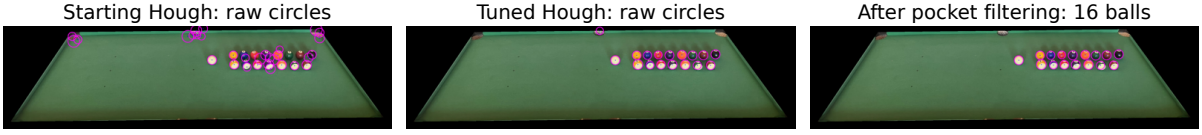


Figure 3: Starting Hough (left), tuned Hough (centre) and pocket filtering (right).

The grid search selected an accumulator ratio of 1.5, minimum centre separation of 18 pixels, upper Canny threshold of 75 and centre-accumulator threshold of 35. Radii were fixed at 15—30 pixels.

Since the Hough method already includes Canny, a separate edge-detection step was not needed. The lower Canny threshold allowed weaker edges to be used, while the accumulator threshold required enough votes before accepting a centre. The minimum centre distance reduced duplicate detections without excluding nearby balls. A ratio of 1.5 used a smaller grid to count votes, saving memory but making centre positions less precise.

Finally, each detected candidate was extracted as a square centred on its location, with side length $2r + 1$. A circular mask excluded the square's corner pixels. Keeping the original colours and mask allowed me to compare recognition approaches on the same crop and inspect individual mistakes.

2.2 Ball Type Recognition

I separated recognition into estimating type and body colour. My first type estimator used HLS lightness and saturation to count white, dark and coloured pixels inside each circle. The proportions then determined the type. But, reflections, shadows and imperfect boundaries made these fixed thresholds unreliable: only 43/86 balls were typed correctly.

I then used K-means to separate light and coloured regions. I initially chose $k = 2$ to reflect the light and coloured regions visible on pool balls, and retained it after testing other values on AllBalls because this simple grouping worked well. I then manually tested the cluster-classification thresholds, retaining $V \geq 170$ and $S \leq 165$ for light; remaining clusters were dark if $V \leq 90$, and coloured otherwise. The light-pixel fraction w determined type: at least 85% light meant White, at least 70% non-light meant Solid, and other mixtures meant Striped. This improved type recognition to 76/86. Higher k could separate shadows and reflections into additional clusters, but would need further evaluation.

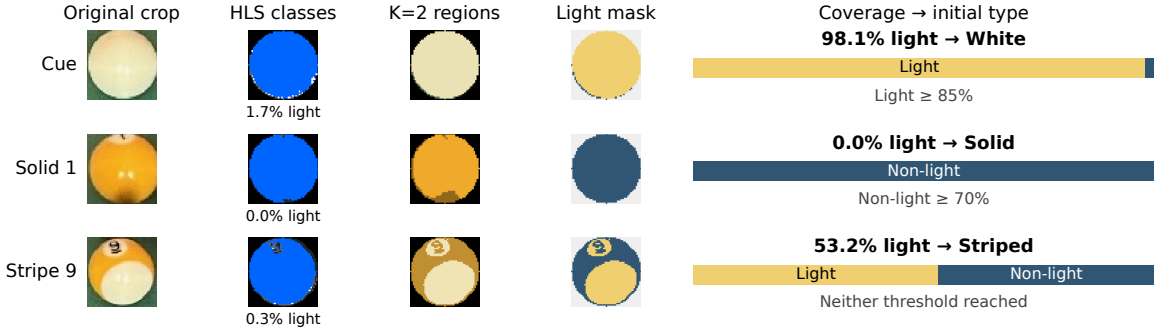


Figure 4: AllBalls cue, solid and stripe examples for HLS and k means

2.3 Ball Colour Recognition

I first applied HSV threshold rules to the mean colour of the largest non-light K-means cluster. But shadows and reflections could distort this single average, so I switched to histograms to preserve the different body colours and their proportions.

I built eight references by grouping labelled AllBalls crops by body colour, pairing solids and stripes. Black is always solid and the cue was excluded. I eroded each circular mask with a 3×3 ellipse and removed pixels with $V \geq 170$, $S \leq 165$. This reduced cloth contamination, shared white stripes, patches and highlights that could make different colours look alike.

For histogram resolutions I selected 18 hue bins and 8 bins each for saturation and brightness, balancing colour discrimination with the limited number of pixels available in each ball crop. I smoothed neighbouring bins with $[1, 2, 1]/4$ along each axis, wrapping hue. Nearly neutral pixels (BGR range below 15) received hue zero.

I normalised each crop's histogram to sum to one before averaging colour partners, giving both equal influence despite different body-pixel counts. Fewer than 12 body pixels gave an Unknown colour label.

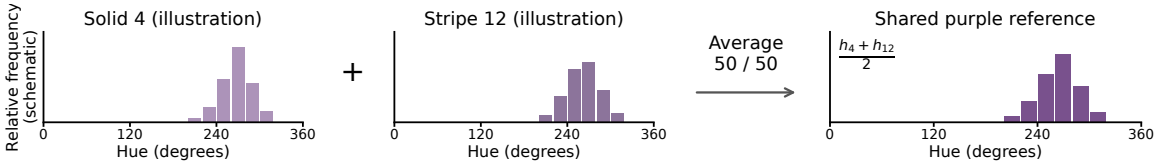


Figure 5: Approximate histogram reference example on purple balls

Finally, each candidate's body-colour histogram were compared with all eight references using one minus Bhattacharyya distance to give similarity $s_{i,c}$ scores. The inital k means correctly classified body colour for 65/80 matched balls. Histogram matching improved this to 73/80 with 8 corrections and no regression.

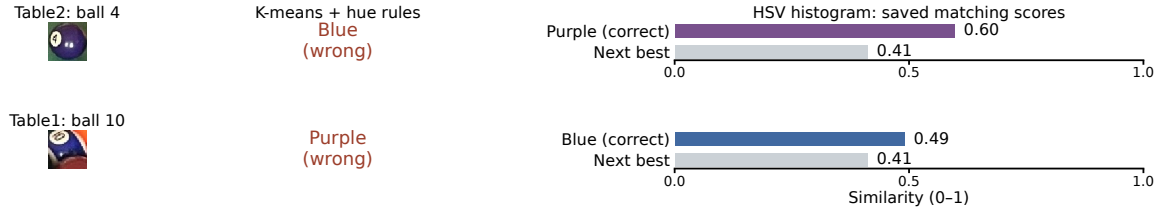


Figure 6: Exported results (csv & ball crops) show two colour corrections.

2.4 Soft Score Approach

Then, a property I wanted to leverage was that each ball has a unique identity. To use this I pivoted to soft scores instead of hard decisions. Therefore, I first replaced the single type classification with compatibility scores for Solid, Striped and Cue. I kept the original light-coverage intervals that were previously used: $[0, 0.30]$ for Solid, $[0.30, 0.85]$ for Striped and $[0.85, 1]$ for White. A type received a score of one inside its interval, with support decreasing gradually outside it: $g_t(w) = \exp\left[-\frac{d(w,I_t)^2}{2(0.10)^2}\right]$

Here $d(w, I_t)$ is the distance from light coverage w to the type's interval, with zero distance inside it. I used a Gaussian fall-off so small deviations beyond a boundary retained strong support, while larger deviations were penalised more heavily [3, 4]. This kept plausible alternatives available for unique assignment. I chose the width 0.10 (ten percentage points) heuristically. It should be noted that these scores measured compatibility, not probability.

For numbered balls, I multiplied histogram colour similarity by type compatibility: $S_{i,n} = s_{i,c(n)} g_{t(n)}(w_i)$. The cue instead used $S_{i,0} = w_i g_{\text{White}}(w_i)$, rewarding high light coverage and adding a penalty below 85%. These scores provided the alternatives needed for the unique assignment stage.

2.5 Unique identity Assignment

To enforce uniqueness, I used the Hungarian algorithm [5] to maximise the total compatibility score across the table. Each row of the score matrix represented a candidate and each column a possible identity. The algorithm built an assignment while allowing earlier pairings to be rearranged, selecting one identity per candidate and using each identity at most once.

Table2 crop	Score for 4	Score for 12	Assigned identity
Purple solid	**0.597**	0.128	4
Purple stripe	0.639	**0.637**	12

Table 1: Scores example; the two purple crops in Table2

In the table, both crops independently preferred 4. However, assigning the stripe to 12 barely reduced its score, whereas assigning the solid to 12 caused a much larger loss. The selected pairing therefore scored 1.234 overall, compared with 0.767 for the reverse pairing.

Each detection also had an Unknown option, allowing surplus or ineligible candidates to remain unassigned. However, Unknown had a near-zero score, so even a weak match could receive an unused identity. This favoured assigning labels wherever possible, but offered limited rejection of false detections. Across Tables1–6, this approach improved correct identities from 73/86 to 75/86.



Figure 7: Cropped overhead view of AllBalls after recognition and perspective correction. AllBalls supplied tuning and colour references.

2.6 Resolving cue-ball ambiguity

Inspection revealed that Hough sometimes found the white patch inside the ball instead of the full ball boundary. Several candidates could then appear White, and uniqueness alone could give the cue to the wrong one. I introduced two refinements to resolve these issues.

First, I expanded the radius of originally White crops by one pixel, which performed best among the padding sizes tested, and recomputed their type and colour evidence. This could recover a thin rim of body colour when detection captured mainly a white patch. The stored centre and original detected radius were unchanged. This improved correct identities from 75/86 to 79/86.

Second, when several original candidates remained classified as White, I reserved the cue for the largest original radius. A full ball should usually exceed a patch: Table1's radii were 24.15 and 17.55 pixels. This improved performance to 80/86. A more sophisticated approach could account for perspective by comparing each candidate's radius with the expected apparent ball radius at its position on the table.

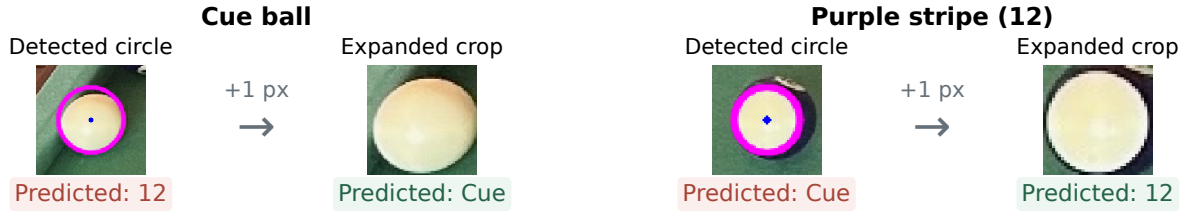


Figure 8: Table3: one-pixel radius expansion corrects the swapped cue and stripe-12 predictions.

3 Results and Analysis

3.1 Evaluation methodology

I evaluated detection and recognition separately to distinguish missed balls from incorrect identities. For detection, I matched predicted centres to annotated positions in the perspective-corrected table, allowing a maximum Euclidean distance of 15 table units. Matching found as many valid pairs as possible, using each detection and annotation only once. Matched pairs counted as true positives (TP), extra detections as false positives (FP), and missed balls as false negatives (FN).

The 15-table-unit matching tolerance was a fixed heuristic, approximately one-third of the assumed 43-unit ball diameter. It allowed some uncertainty in detection and annotation while requiring the predicted centre to remain within the ball's nominal radius.

Precision, TP/(TP+FP), measured how many detections corresponded to real balls. Recall, $\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$, measured how many annotated balls were found. F1 combined them as $2\mathrm{TP}/(2\mathrm{TP}+\mathrm{FP}+\mathrm{FN})$. Mean localisation error measured the average centre-to-centre distance of matched pairs using MLE $= \frac{1}{N}\sum_{i=1}^{N}\sqrt{(x_i-\hat{x}_i)^2+(y_i-\hat{y}_i)^2}$.

Recognition accuracy measured the fraction of matched balls given the correct identity. End-to-end accuracy divided correct identities by all annotated balls, so missed balls also counted as failures. Unknown labels counted as incorrect. False detections lowered detection precision but were excluded from recognition accuracy because they don't match a real ball whose identity could be assessed. Body-colour accuracy excluded the cue.

I reported AllBalls separately because it supplied tuning and colour references. The overall results combine Tables1–6, which also guided development. They therefore describe performance on images used to refine the system, rather than an independent test.

3.2 Final Metrics

The final system detected 86 of the 90 annotated balls, with two false positives. Precision was **97.73%**, recall **95.56%** and F1 **96.63%**. Mean centre error was 4.91 table units across the 86 matches. Of those matched balls, **80/86** had the correct identity (**93.02%**), giving **80/90**, or **88.89%**, end-to-end accuracy.

I also compared performance for each ball identity to see which balls were recognised reliably and which caused the most errors.

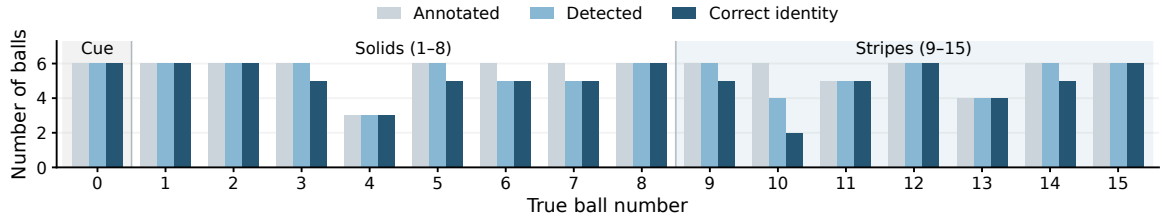


Figure 9: Counts by true identity on Tables1–6 (0 is the cue). Correct identities: solids 41/45, stripes 33/39, cues 6/6.

3.3 Final Table Views

Using the supplied corners, I calculated a perspective transform that mapped the playing surface to a 1434×683 rectangle, then applied the same transform to the detected ball centres. This was done post detection and recognition so as to not affect the ball shapes.

Table1: TP 14 | FP 1 | FN 2
IDs 11/16 | mean centre error 7.68

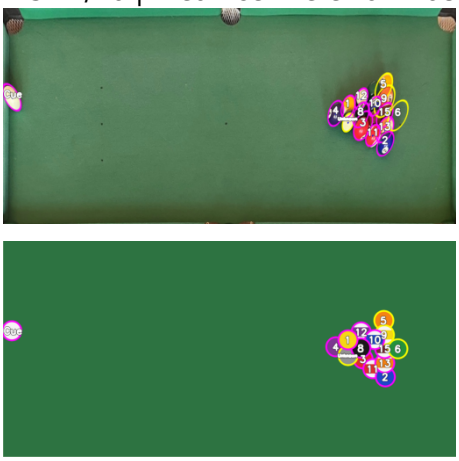


Table2: TP 16 | FP 1 | FN 0
IDs 16/16 | mean centre error 3.58

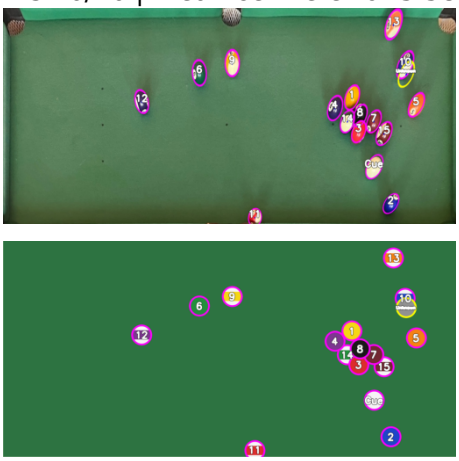


Table3: TP 15 | FP 0 | FN 1
IDs 15/16 | mean centre error 3.12

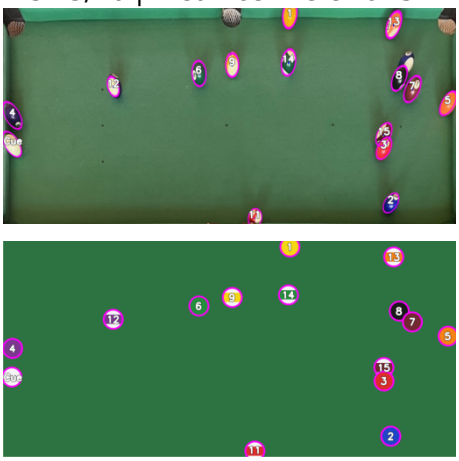
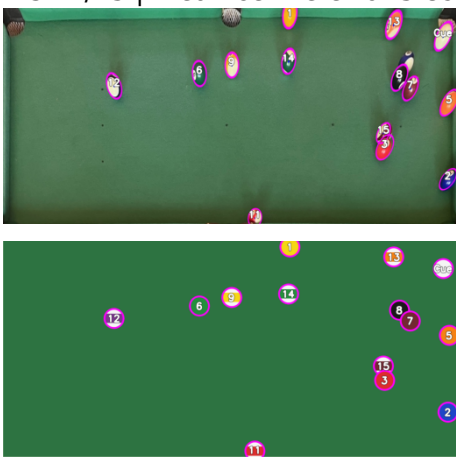


Table4: TP 14 | FP 0 | FN 1
IDs 14/15 | mean centre error 3.89



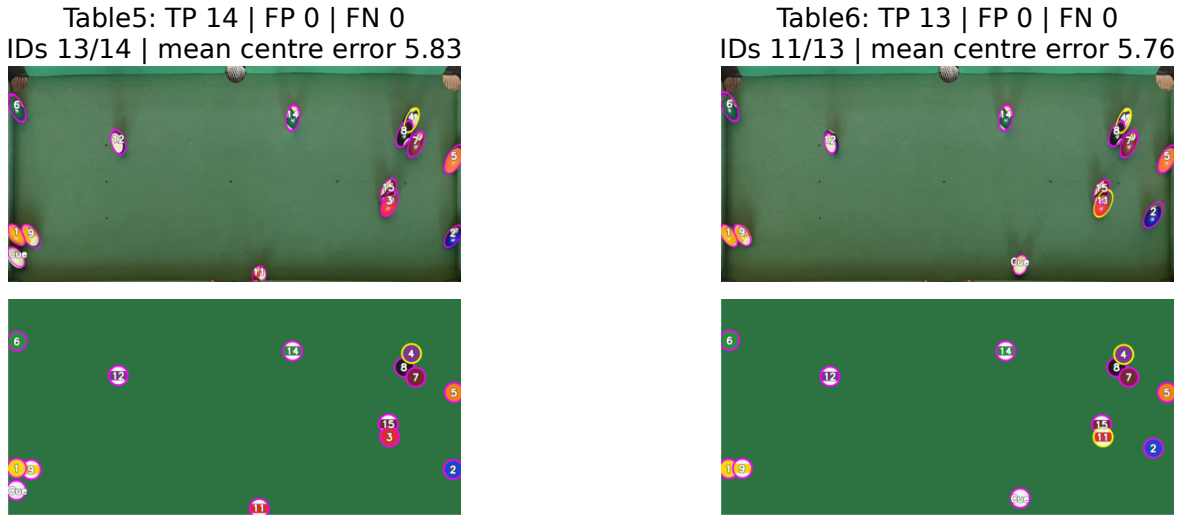


Figure 10: Final visualisation for Tables1–6.

The final views pair a numbered rectified photograph with an overhead diagram. The photograph shows the predictions in context, while the diagram uses enlarged markers for readability. Magenta outlines indicate correct identities while yellow indicates wrong identities or false circles. These outline colours were added after prediction by comparing results with the ground truth.

3.4 Observed Strengths and Weaknesses

Table2 was the clearest success: all 16 real balls were correctly identified, while the extra circle remained Unknown. The system could therefore handle some ambiguous appearances and an additional detection without losing correct identities. Tables3 and 4 also identified every detected ball correctly, although both missed ball 10. In these cases, detection was the remaining obstacle to a completely correct result.

Table1 exposed the limits of circle detection in a crowded arrangement. Balls 6 and 7 were missed, a false circle received identity 6, and balls 5 and 9 exchanged labels. The exchanged balls had centre errors of 14.63 and 14.81 table units, close to the 15-unit matching tolerance. They counted as successful detections, but their displaced crops could include neighbouring material or exclude part of the intended ball. A valid centre match therefore did not guarantee useful recognition evidence.

In Table1, the green stripe's crop captured mainly white, leaving too little green for matching. One-pixel padding performed best overall among the sizes tested, but this ball remained Unknown. Table6's blue stripe instead included a dark neighbour and was labelled as a purple solid. Larger crops may recover missing colour, but can also admit neighbours, so increasing padding alone cannot reliably resolve both errors.

Table6's red solid exposed a separate weakness. Its bright red surface passed the light thresholds, inflating white coverage and producing a red-striped label despite the correct body-colour match.

The system performed well overall, but the remaining errors show there is still room to improve detection and recognition.

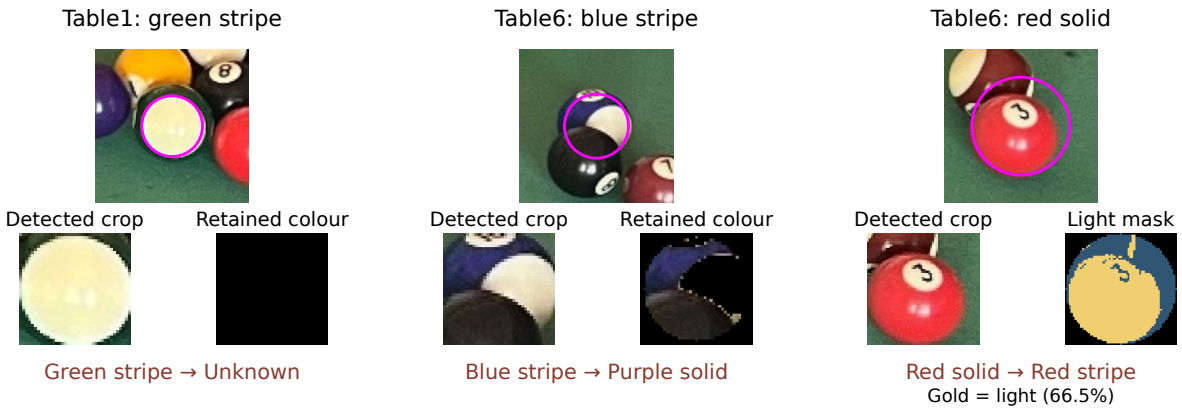


Figure 11: Failure examples: context and detected circles above, crops and evidence below. Gold marks pixels counted as light; black marks excluded pixels.

3.5 Possible Next Steps & Conclusion

The current type classifier uses k-means colour groups and their proportions, without considering their spatial arrangement. Connected-component analysis (CCA) could examine the size, position and connectivity of white and coloured regions, adding evidence to help distinguish solids from stripes.

For detection, guided Hough could focus circle searches on crowded regions using the expected ball size. Watershed could then use the candidate centres as markers to separate touching balls into individual regions. This could help recover missed balls and produce cleaner recognition crops containing less neighbouring material.

Additionally, we could test lighting correction to make white coverage estimates more accurate and consistent across different lighting conditions.

Conclusion. The pipeline performed well for the tested setup. AllBalls provided clear reference examples but had limited variety, so the different arrangements in Tables1–6 helped reveal more difficult cases such as overlapping balls. I focused on changes that addressed recurring errors across the images, aiming to keep the approach applicable beyond individual cases. To fully assess the system's performance and ability to generalise, testing on additional unseen images with the same fixed parameters would help establish how well it handles new ball arrangements and lighting conditions.

References

[1] K. Dawson-Howe, *CSU44053 Computer Vision course material*, Trinity College Dublin.

[2] OpenCV, *OpenCV 4.12.0 Documentation*

[3] Scikit-learn Developers, *User Guide*, §1.7.4.3, "Radial basis function (RBF) kernel."

[4] MathWorks, *Fuzzy Logic Toolbox documentation*, "Gaussian combination membership function."

[5] CP-Algorithms, "Hungarian Algorithm."

Code Repository: `https://github.com/galiannm/pool-ball-recogniser`